Plan de Game Day — Chaos Engineering

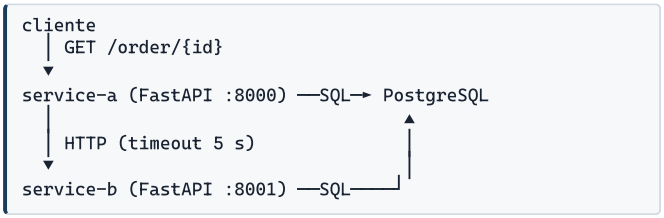
Hipótesis de fallo, experimentos controlados y análisis de resiliencia sobre un pipeline de observabilidad con OpenTelemetry

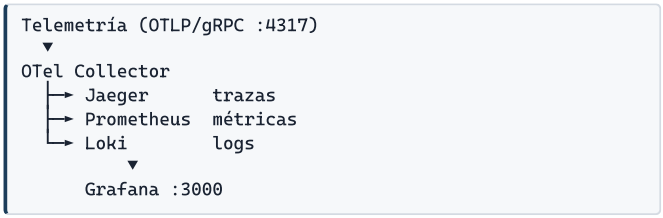
Programa: Maestría en Arquitectura de Software — Universidad de La Sabana · **Asignatura:** Observabilidad en Ambientes Productivos · **Fecha:** 15/10/2026

Integrantes: Edgar Andrés Romero Otálora · Jhon Alexander Tenjo Romero · Julián David Miranda Leguizamón

1. Sistema objetivo

El objetivo es el pipeline de observabilidad construido en el laboratorio anterior: dos microservicios FastAPI instrumentados con el SDK de OpenTelemetry, con una dependencia HTTP entre ellos y acceso a base de datos, cuya telemetría se centraliza en un OTel Collector.





Dependencias críticas: service-a depende de service-b para enriquecer cada pedido con datos de inventario; ambos dependen de PostgreSQL; y toda la observabilidad depende del Collector. Cada contenedor está limitado a **1.0 CPU y 512 MiB**, lo que acota el radio de impacto de cualquier experimento de agotamiento de recursos.

1.1 Estado estable medido

Las hipótesis no parten de supuestos: el estado estable se midió sobre el sistema en funcionamiento antes de diseñar los experimentos.

SLI	Valor en estado estable	Observación
Latencia p50	50,1 ms	Flujo completo A → DB → B → DB
Latencia p95	115,4 ms	
Latencia p99	360,3 ms	Cola lenta, afectada por el jitter de service-b
Disponibilidad	100,0 %	Sin errores 5xx
CPU service-b	5,5 %	Sobre una cuota de 1.0 CPU
CPU service-a	4,9 %	

2. Hipótesis de fallo

H1 Latencia de red CONFIRMADA

En estado estable (p99 = 360 ms, disponibilidad 100 %), **cuando** se inyectan 200 ms de latencia de red en el camino hacia service-b, **esperamos que** la latencia p99 de service-a aumente aproximadamente en esa cantidad pero la disponibilidad se mantenga en 100 %, porque el timeout de service-a hacia service-b es de 5 s y conserva margen suficiente para absorber el retraso.

H2 Agotamiento de CPU CONFIRMADA

En estado estable (CPU de service-b al 5,5 %, p95 = 115 ms), **cuando** se satura por completo la cuota de CPU de service-b, **esperamos que** la latencia p95 se degrade y el throughput caiga, pero que **no aparezcan errores 5xx**, porque la degradación proviene de contención de recursos y no de un fallo de la aplicación: las peticiones siguen completándose, sólo que más despacio.

H3 Partición de red REFUTADA PARCIALMENTE

En estado estable (disponibilidad 100 %), **cuando** service-b deja de ser alcanzable por completo, **esperamos que** service-a siga respondiendo HTTP 200 con `inventory.available = -1` —degradación elegante— en lugar de propagar un 5xx, y que el contador `service_b_calls_total{result="unavailable"}` registre cada fallo.

3. Diseño de los experimentos

Parámetro	E1 — Latencia	E2 — CPU	E3 — Partición
Tipo de fallo	network latency	resource exhaustion	network partition
Radio de impacto	Camino service-a → service-b. PostgreSQL y el stack de observabilidad intactos	Contenedor service-b, acotado por su cuota de 1.0 CPU	La dependencia A → B. La telemetría del incidente se sigue recogiendo
Duración	3 min	3 min	2 min
Justificación	Suficiente para que el `rate()` de Prometheus (ventana de 1 min) estabilice las métricas y para medir el MTTD, pero corto para acotar la exposición. E3 se limita a 2 min por ser el más disruptivo: deja una dependencia completamente caída.		
Rollback	`DELETE` del toxic, en un `trap` de salida	`pkill stress-ng` en `trap` + `--timeout` propio del proceso	`docker network connect` en `trap`, con variable de estado
Herramienta	Toxiproxy	stress-ng	docker network disconnect
Equivalente en K8s	`NetworkChaos` action: delay	`StressChaos` stressors.cpu	`NetworkChaos` action: partition

Seguridad de los experimentos. Ninguno se ejecuta sin antes verificar el estado estable: si el sistema ya está degradado el script **aborta**, porque medir sobre un sistema enfermo no permite atribuir el impacto al fallo inyectado. Todos los rollbacks están en un `trap`, de modo que se ejecutan aunque el script falle o se interrumpa con Ctrl+C. E2 lleva además un segundo mecanismo independiente (`--timeout`): si el script muriera, el estresador se detiene solo.

3.1 Métricas de observabilidad para verificar cada hipótesis

Exp.	SLI que mide el impacto	SLI de control	Logs y trazas que confirman el comportamiento
E1	Latencia p99 y p95 `http_request_duration_seconds`	Disponibilidad (debe seguir en 100 %)	Traza en Jaeger con el span `call.service-b.inventory` alargado mientras el resto del árbol no cambia
E2	CPU del contenedor `container_cpu_utilization_ratio`	Tasa de errores 5xx (debe seguir en 0)	Panel de CPU por contenedor; histograma de latencia desplazándose sin aparición de errores
E3	Contador de degradación `service_b_calls_total{result="unavailable"}`	Disponibilidad de service-a (no debe propagar 5xx)	Logs `service-b inalcanzable` en Loki; spans `call.service-b.inventory` en estado ERROR

4. Ejecución y resultados

Los tres experimentos se ejecutaron en el entorno local con Docker Compose, siguiendo el modelo de *probes* de LitmusChaos: verificación del estado estable (SOT), inyección, medición de la detección (DuringChaos), rollback y verificación de la recuperación (EOT).

Desviación respecto al plan inicial. E1 iba a usar `tc netem`, pero no es viable en este entorno: el kernel de Docker Desktop sobre WSL2 no incluye el módulo `sch_netem` y `/lib/modules` está vacío, por lo que tampoco puede cargarse (`Specified qdisc kind is unknown`). Se sustituyó por **Toxiproxy**, que consigue el mismo efecto observable —latencia real en el transporte, inyectada *fuera* de la aplicación— sin depender de módulos del kernel. La propiedad que importa se conserva: ninguna línea de código de los servicios sabe que hay un experimento en curso. `stress-ng` (E2) sí se usó tal cual.

4.1 Resultados esperados frente a obtenidos

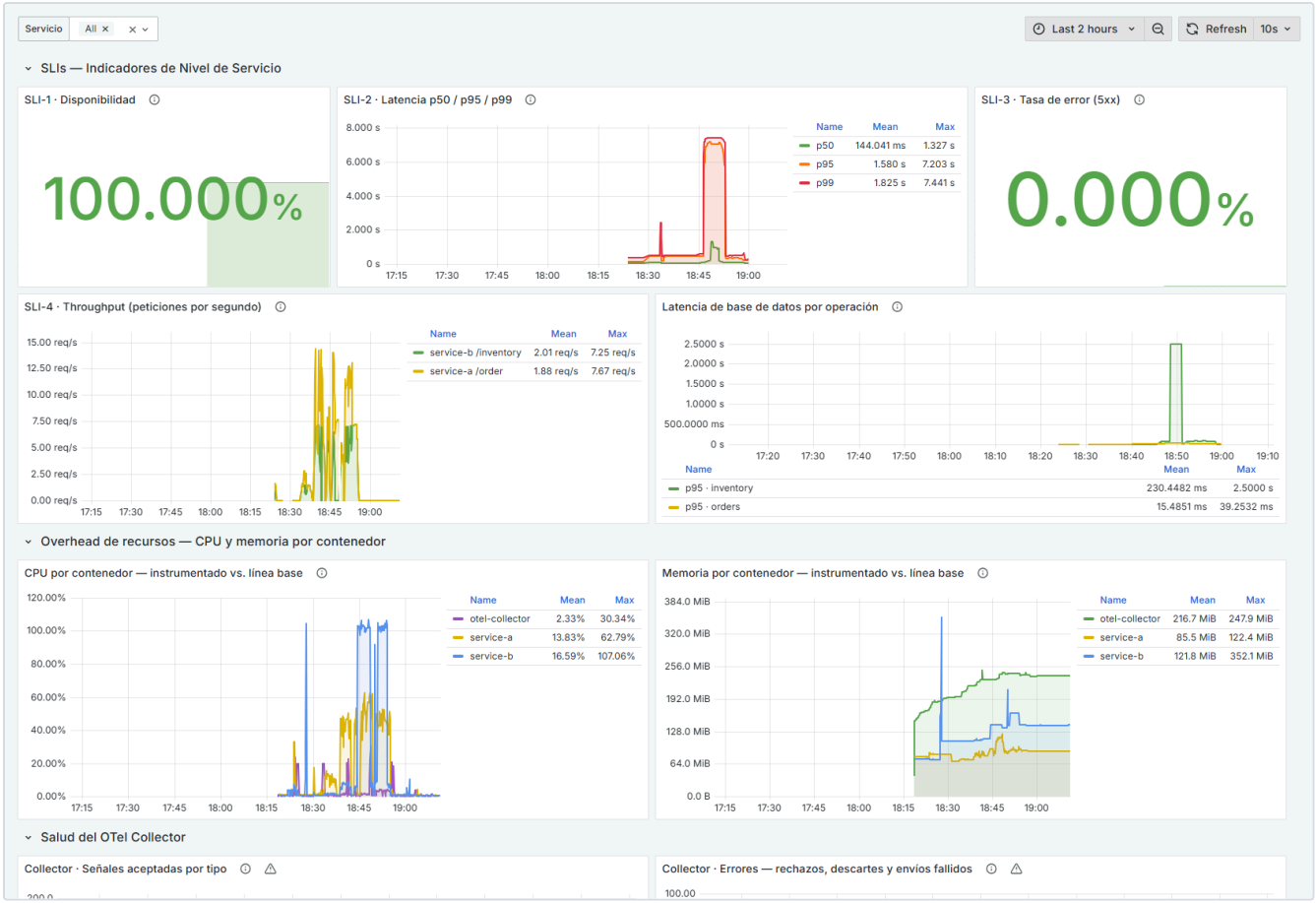
Exp.	SLI	Base	Con fallo	Δ	Esperado vs. obtenido
E1	Latencia p99	366.5 ms	551.2 ms	+50 %	Se esperaba un aumento cercano a los 200 ms inyectados y disponibilidad intacta. **El incremento medido fue de 184.7 ms**, y la disponibilidad se mantuvo. La hipótesis se cumple con precisión.
	Latencia p95	212.6 ms	480.7 ms	+126 %	
	Disponibilidad	100.0 %	**100.0 %**	=	
E2	CPU service-b	9.1 %	**103.7 %**	+1035 %	La CPU superó el 100 % de su cuota, la latencia se degradó y **no apareció ni un solo error 5xx**. Confirma que el sistema se vuelve lento, no indisponible, cuando le falta CPU.
	Latencia p95	229.7 ms	412.0 ms	+79 %	
	Latencia p99	389.9 ms	969.7 ms	+149 %	
	Disponibilidad	100,0 %	**100.0 %**	=	
E3	Latencia p99	481.2 ms	**7.475.0 ms**	**+1453 %**	El fallback **sí** actuó (836 degradaciones registradas, sin 5xx), pero a un coste inaceptable: **el p99 se disparó a 7.475.0 ms**. La hipótesis se cumple en la forma pero se refuta en la práctica.
	Degradaciones	0	836	—	
	p99 tras rollback	—	**436.0 ms**	recuperado	

4.2 Tiempos de detección y recuperación

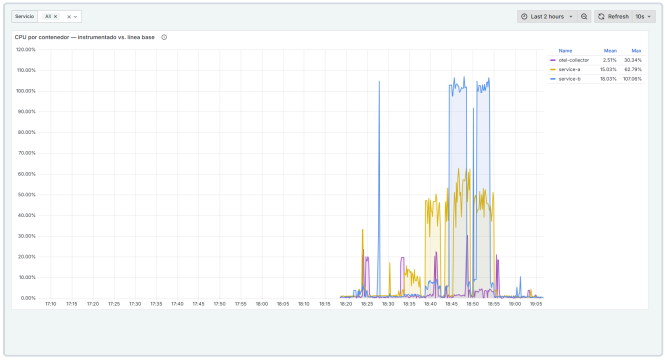
Experimento	MTTD	Recuperación (EOT)	Comentario
E1 — Latencia	61 s	1 s	La detección depende de la ventana de `rate()`: con 1 min, el percentil tarda ese tiempo en reflejar el cambio
E2 — CPU	**0 s**	39 s	Inmediata: `docker_stats` expone la CPU cada 10 s y el pico es instantáneo
E3 — Partición	**16 s**	56 s	El contador de degradación reacciona mucho antes que cualquier percentil de latencia

> **Hallazgo sobre la propia observabilidad.** El MTTD varía un orden de magnitud según qué señal se vigile: 16 s con un contador de eventos frente a 61 s con un percentil de latencia. Para alertar sobre dependencias caídas conviene vigilar **contadores de eventos discretos**, no agregados estadísticos, porque un percentil necesita acumular muestras antes de moverse.

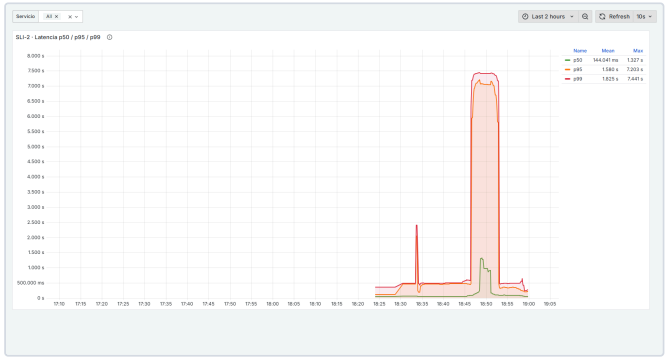
5. Evidencias



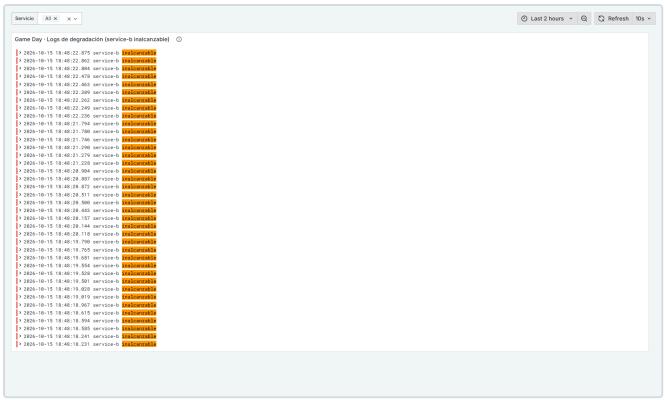
Dashboard durante el Game Day. Los cuatro SLIs, el consumo de recursos por contenedor y la salud del Collector a lo largo de los tres experimentos. Los picos sucesivos de latencia y CPU corresponden, en orden, a E1, E2 y E3.



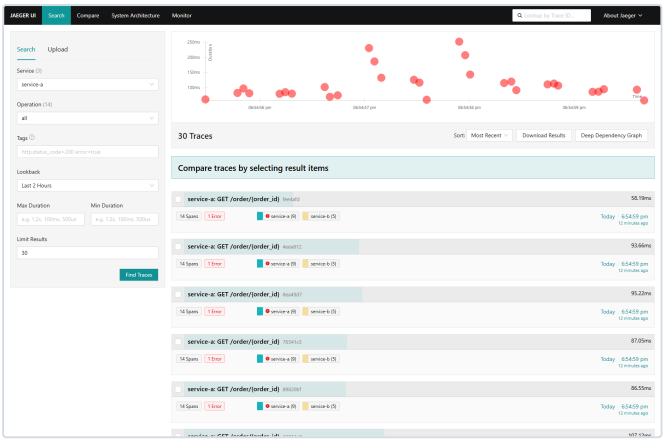
E2 — CPU saturada. service-b alcanza el **107 % de su cuota** de 1.0 CPU mientras `stress-ng` está activo. Las dos mesetas corresponden a las dos ejecuciones del experimento.



Percentiles de latencia. Los picos de p95 y p99 durante los experimentos, y el retorno a la línea base tras cada rollback.



E3 — Degradación registrada. Logs `service-b inalcanzable` durante la partición. Cada línea lleva su `trace_id`, de modo que desde el log se salta a la traza del request que sufrió el fallo: la prueba de que el servicio degradó en lugar de propagar el error.



Trazas en Jaeger. Junto a las normales aparecen las de varios segundos provocadas por el timeout durante E3.

6. Debilidad sistémica revelada

El experimento E3 reveló una debilidad que no era visible en operación normal ni en las pruebas de carga del laboratorio anterior:

> **El timeout de 5 s convierte una dependencia caída en una latencia de 7.475.0 ms.** El mecanismo de degradación funciona —service-a nunca devolvió un 5xx y registró 836 degradaciones—, pero *cada petición espera el timeout completo antes de degradar*. Desde la perspectiva del usuario, una respuesta correcta que tarda más de siete segundos es indistinguible de un servicio caído.

El efecto es acumulativo: mientras cada petición ocupa un hueco de concurrencia durante 5 s, las siguientes se encolan. Durante E3 llegaron a completarse tan pocas respuestas que las métricas de throughput y disponibilidad se quedaron sin muestras, lo que en sí mismo es un síntoma: **el sistema dejó de poder informar sobre su propio estado**. La causa de fondo es que **el código trata un fallo total como si fuera lentitud puntual**: un timeout generoso tiene sentido ante lentitud esporádica, pero ante una dependencia caída sólo propaga el daño hacia arriba.

7. Remediaciones propuestas

#	Acción	Efecto esperado	Esfuerzo
1	**Reducir el timeout de 5 s a 1 s.** El p99 en estado estable es de 360 ms, así que 1 s mantiene un margen de casi 3× sobre el peor caso normal	Recorta la espera por petición en un 80 %: el p99 bajo fallo pasaría de ~7,5 s a ~1,2 s	Trivial: una variable de entorno (`SERVICE_B_TIMEOUT`)
2	**Circuit breaker.** Tras N fallos consecutivos, abrir el circuito y degradar de inmediato sin intentar la llamada, con reintentos de prueba espaciados	Elimina la espera por completo mientras la dependencia esté caída: el p99 volvería a valores cercanos a la línea base (~500 ms)	Medio: añadir una librería de resiliencia y exponer su estado como métrica
3	**Alerta sobre el contador de degradación.** Disparar cuando `service_b_calls_total{result="unavailable"}` crezca sobre su línea base	La señal ya existe y reaccionó en 16 s; falta convertirla en aviso accionable	Bajo: una regla de alerta en Prometheus
4	**Presupuesto de latencia de extremo a extremo.** Que el timeout hacia una dependencia se derive del SLO del endpoint, en lugar de fijarse por intuición	Evita que vuelva a aparecer la misma clase de problema en futuras dependencias	Medio: decisión de diseño, no de código

8. Conclusiones

De las tres hipótesis, dos se confirmaron y una se refutó parcialmente. El valor del ejercicio está precisamente en la tercera: **las hipótesis que se cumplen confirman lo que ya se creía saber, mientras que la que falla enseña algo nuevo**. E1 y E2 demostraron que el sistema se comporta como se esperaba ante latencia y falta de CPU; E3 destapó que el mecanismo de degradación, aun funcionando, tiene un coste que lo vuelve ineficaz en la práctica.

El segundo aprendizaje es sobre la observabilidad misma. Las tres señales detectaron los fallos, pero con tiempos muy distintos: un contador de eventos delató la partición de red en 16 s, mientras que un percentil de latencia tardó 61 s en reflejar la degradación de E1. Al diseñar alertas, la elección de la señal importa tanto como el umbral.

Por último, el ejercicio confirmó el principio central del Chaos Engineering: **el valor no está en provocar fallos, sino en la hipótesis previa**. Sin haber medido el estado estable y escrito qué se esperaba, el p99 de 7.475.0 ms de E3 habría parecido simplemente "el sistema va lento"; con la hipótesis delante, se identificó como una debilidad concreta —el timeout— con una remediación concreta y medible.